

Name: Santiago, Jalene	Date Performed: December 11, 2025
Course/Section: CPE212 - CPE31S1	Date Submitted: December 11, 2025
Instructor: Engr. Robin Valenzuela	Semester and SY: 2nd Sem 2025-2026
Activity 2: SSH Key-Based Authentication and Setting up Git	
1. Objectives: 1.1 Configure remote and local machine to connect via SSH using a KEY instead of using a password 1.2 Create a public key and private key 1.3 Verify connectivity 1.4 Setup Git Repository using local and remote repositories 1.5 Configure and Run ad hoc commands from local machine to remote servers	
Part 1: Discussion It is assumed that you are already done with the last Activity (Activity 1: Configure Network using Virtual Machines). <i>Provide screenshots for each task.</i> It is also assumed that you have VMs running that you can SSH but requires a password. Our goal is to remotely login through SSH using a key without using a password. In this activity, we create a public and a private key. The private key resides in the local machine while the public key will be pushed to remote machines. Thus, instead of using a password, the local machine can connect automatically using SSH through an authorized key. What is ssh-keygen? Ssh-keygen is a tool for creating new authentication key pairs for SSH. Such key pairs are used for automating logins, single sign-on, and for authenticating hosts. SSH Keys and Public Key Authentication The SSH protocol uses public key cryptography for authenticating hosts and users. The authentication keys, called SSH keys, are created using the keygen program. SSH introduced public key authentication as a more secure alternative to the older .rhosts authentication. It improved security by avoiding the need to have passwords stored in files and eliminated the possibility of a compromised server stealing the user's password. However, SSH keys are authentication credentials just like passwords. Thus, they must be managed somewhat analogously to usernames and passwords. They should have a proper termination process so that keys are removed when no longer needed.	
Task 1: Create an SSH Key Pair for User Authentication 1. The simplest way to generate a key pair is to run <i>ssh-keygen</i> without arguments. In this case, it will prompt for the file in which to store keys. First,	

the tool asked where to save the file. SSH keys for user authentication are usually stored in the users `.ssh` directory under the home directory. However, in enterprise environments, the location is often different. The default key file name depends on the algorithm, in this case `id_rsa` when using the default RSA algorithm. It could also be, for example, `id_dsa` or `id_ecdsa`.

2. Issue the command `ssh-keygen -t rsa -b 4096`. The algorithm is selected using the `-t` option and key size using the `-b` option.
3. When asked for a passphrase, just press enter. The passphrase is used for encrypting the key, so that it cannot be used even if someone obtains the private key file. The passphrase should be cryptographically strong.
4. Verify that you have created the key by issuing the command `ls -la .ssh`. The command should show the `.ssh` directory containing a pair of keys. For example, `id_rsa.pub` and `id_rsa`.

TASK 1 SCREENSHOTS

```
jalene@workstation:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/jalene/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/jalene/.ssh/id_ed25519
Your public key has been saved in /home/jalene/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:yerQ07kZUDIGmp9ZZ0FZgcHwc+LdfJgGZKaeFyXrCqk jalene@workstation
The key's randomart image is:
+--[ED25519 256]--+
| oo+=B..          |
| o *oB +          |
|o  % *            |
| . O @ * +        |
| * = + S .        |
| . . = o .        |
|E  o +.           |
|    oo+           |
|    =o            |
+-----[SHA256]-----+
```

```

jalene@workstation:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/jalene/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/jalene/.ssh/id_rsa
Your public key has been saved in /home/jalene/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:gmq0XuxCL/KrQa8IRZdanQich76Ej59ip4hCMY36e68 jalene@workstation
The key's randomart image is:
+---[RSA 4096]-----+
| ..o                |
| +..+ .             |
| o+= o              |
| .=o= .             |
| .=.. . S           |
| +=. .              |
| =o+=               |
| 00Bo.              |
| @BX=Eo.            |
+-----[SHA256]-----+

```

```

jalene@workstation:~$ ls -la .ssh
total 32
drwx----- 2 jalene jalene 4096 Dec 11 09:29 .
drwxr-x--- 15 jalene jalene 4096 Dec 11 08:58 ..
-rw----- 1 jalene jalene  0 Dec 11 08:53 authorized_keys
-rw----- 1 jalene jalene  411 Dec 11 09:29 id_ed25519
-rw-r--r-- 1 jalene jalene  100 Dec 11 09:29 id_ed25519.pub
-rw----- 1 jalene jalene 3381 Dec 11 09:29 id_rsa
-rw-r--r-- 1 jalene jalene  744 Dec 11 09:29 id_rsa.pub
-rw----- 1 jalene jalene 1404 Dec 11 09:15 known_hosts
-rw-r--r-- 1 jalene jalene  142 Dec 11 09:13 known_hosts.old

```

Task 2: Copying the Public Key to the remote servers

1. To use public key authentication, the public key must be copied to a server and installed in an *authorized_keys* file. This can be conveniently done using the *ssh-copy-id* tool.
2. Issue the command similar to this: *ssh-copy-id -i ~/.ssh/id_rsa user@host*
3. Once the public key has been configured on the server, the server will allow any connecting user that has the private key to log in. During the login process, the client proves possession of the private key by digitally signing the key exchange.

4. On the local machine, verify that you can SSH with Server 1 and Server 2. What did you notice? Did the connection ask for a password? If not, why?

The connection did not ask for a password anymore. This is because we generated an rsa key that acted as a password. We also copied the public key to server 1 which allows the local machine to access the server 1 without entering the server 1's password.

TASK 2

```
jalene@workstation:~$ ssh-copy-id -i ~/.ssh/id_rsa jalene@server1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/jalene/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
jalene@server1's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'jalene@server1'"
and check to make sure that only the key(s) you wanted were added.
```

```
jalene@workstation:~$ ssh jalene@server1
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Dec 11 09:15:22 2025 from 192.168.56.101
jalene@server1:~$
```

Reflections:

Answer the following:

1. How will you describe the ssh-program? What does it do?

The SSH program is simply a software which implements the SSH, the protocol that allows secured remote access over unsecured networks.

2. How do you know that you already installed the public key to the remote servers?
When we can already access the remote server without being prompted for a password. Just like what I did on this activity, I generated rsa key pairs and copied it to server1, when I tried to SSH server1, it did not prompt me for a password anymore.

Part 2: Discussion

Provide screenshots for each task.

It is assumed that you are done with the last activity (**Activity 2: SSH Key-Based Authentication**).

Set up Git

At the heart of GitHub is an open-source version control system (VCS) called Git. Git is responsible for everything GitHub-related that happens locally on your computer. To use Git on the command line, you'll need to download, install, and configure Git on your computer. You can also install GitHub CLI to use GitHub from the command line. If you don't need to work with files locally, GitHub lets you complete many Git-related actions directly in the browser, including:

- Creating a repository
- Forking a repository
- Managing files
- Being social

Task 3: Set up the Git Repository

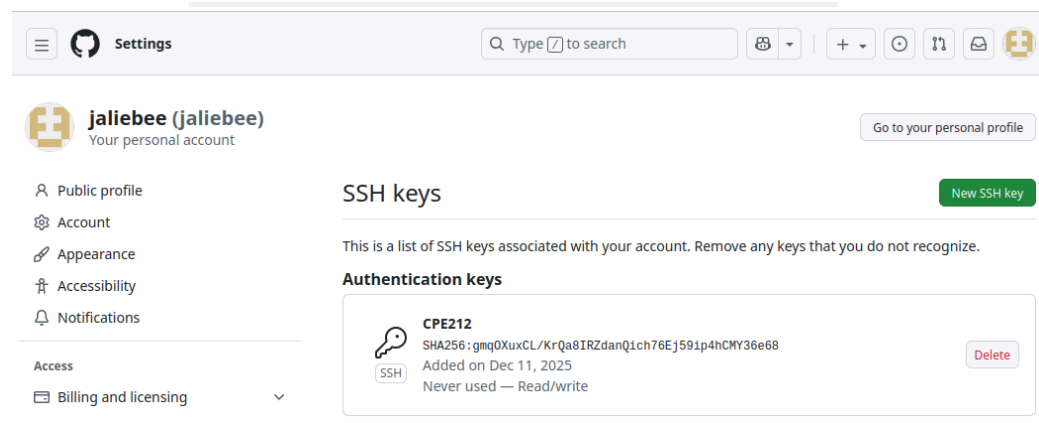
1. On the local machine, verify the version of your git using the command *which git*. If a directory of git is displayed, then you don't need to install git. Otherwise, to install git, use the following command: *sudo apt install git*
2. After the installation, issue the command *which git* again. The directory of git is usually installed in this location: *user/bin/git*.
3. The version of git installed in your device is the latest. Try issuing the command *git --version* to know the version installed.

```
jalene@workstation:~$ git --version
git version 2.43.0
jalene@workstation:~$
```

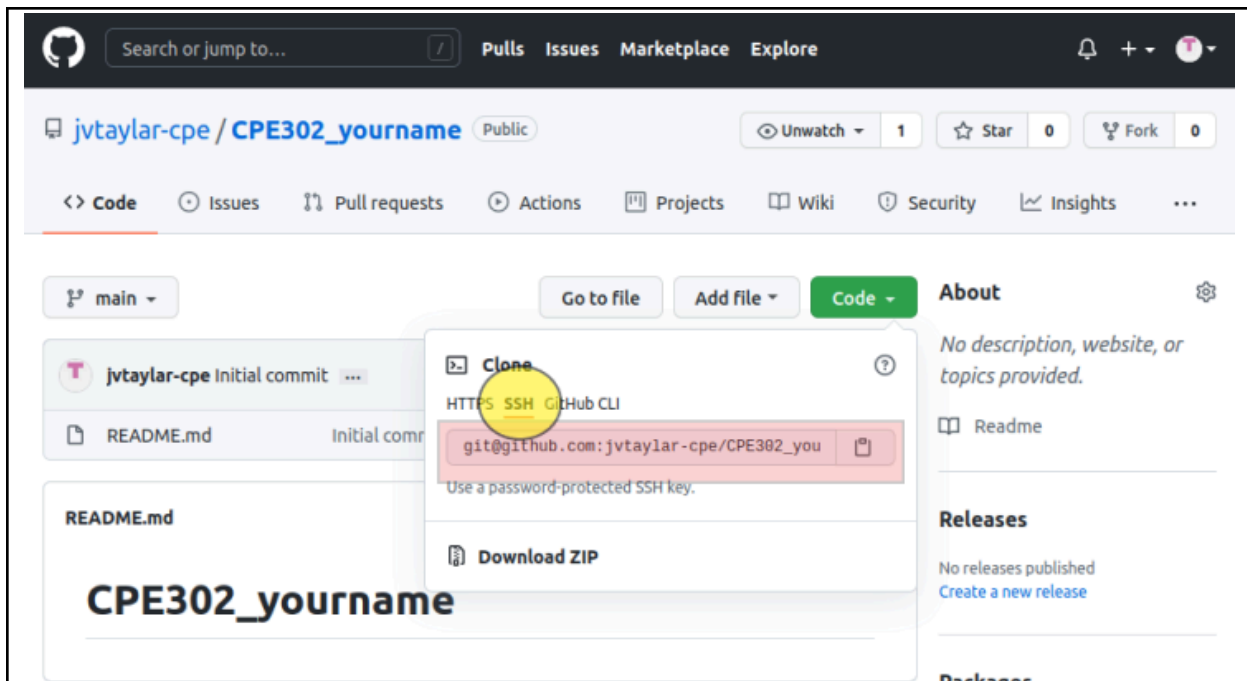
4. Using the browser in the local machine, go to www.github.com.
5. Sign up in case you don't have an account yet. Otherwise, login to your GitHub account.
 - a. Create a new repository and name it as CPE232_yourname. Check Add a README file and click Create repository.

- b. Create a new SSH key on GitHub. Go to your profile's setting and click SSH and GPG keys. If there is an existing key, make sure to delete it. To create a new SSH keys, click New SSH Key. Write CPE232 key as the title of the key.
- c. On the local machine's terminal, issue the command `cat .ssh/id_rsa.pub` and copy the public key. Paste it on the GitHub key and press Add SSH key.

```
jalene@workstation:~$ cat .ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDFA/cjrlIRj+BWxVs/3YFM2+bWQZ40o
n5HIJNzvJntUaNEIjfdNubixtN30CRxodNjFviAwkarWfnLx5uz6FZj8qh22dsV+vR00
QnbUWgKyrBDDFK0oXiCakDdaIESgkgL31vTEFApvXV2guyqM4BVbg8ZoGxDn2mG249m1
a2BGOVa55V6o4lFfshbWm+/j1pSVbhyz43WNjHdruKSYj9TJfIEiou/xuKTno0R0m/mX
Z/hVBf0/wt7a609U/N+TynvhjTEuGzNYyhVDPjilDAWqTfbXR10Ht80ms6Wne3iQEnu
+3N05W9tU29vAx+XqEXWcoGz9mpaLhc9Jstlhkg2IXE8Eoj0IqxczvcSYOfuzk4l001A
GMGn7gtGFERwu+SpjcmrNizjNeFgXaDNSFOv6HItW7A4iggUZxIM4rxA8gLuNQM8UKHb
uwsK12YeIVvEWHaJOvYpIhi3KP8WAYZH8e7nIvGh/llJ6w1iItpsEX/WozK0/MKyLC0a
ghmIHU1+VxdvB6pMIUELETlTRwm24fn5umHs5tWft3DuTlF10iw30/Kw8yL0Y+zvEP4p
ww== jalene@workstation
jalene@workstation:~$
```



- d. Clone the repository that you created. In doing this, you need to get the link from GitHub. Browse to your repository as shown below. Click on the Code drop down menu. Select SSH and copy the link.



- e. Issue the command `git clone` followed by the copied link. For example, `git clone git@github.com:jvtaylor-cpe/CPE232_yourname.git`. When prompted to continue connecting, type yes and press enter.

```
jalene@workstation:~$ git clone git@github.com:jaliebee/CPE212_jalene.git
Cloning into 'CPE212_jalene'...
The authenticity of host 'github.com (20.26.156.215)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
jalene@workstation:~$
```

- f. To verify that you have cloned the GitHub repository, issue the command `ls`. Observe that you have the CPE232_yourname in the list of your directories. Use `CD` command to go to that directory and `LS` command to see the file `README.md`.

```
jalene@workstation:~$ ls
CPE212_jalene  Documents  Music      Public  Templates
Desktop        Downloads  Pictures   snap    Videos
jalene@workstation:~$
```

- g. Use the following commands to personalize your git.
- `git config --global user.name "Your Name"`
 - `git config --global user.email yourname@email.com`

- Verify that you have personalized the config file using the command `cat ~/.gitconfig`

```
jalene@workstation:~$ git config --global user.name "Jalene"
jalene@workstation:~$ git config --global user.email psantiagojalene@gmail.com
jalene@workstation:~$ cat ~/.gitconfig
[user]
    name = Jalene
    email = psantiagojalene@gmail.com
jalene@workstation:~$
```

- h. Edit the README.md file using nano command. Provide any information on the markdown file pertaining to the repository you created. Make sure to write out or save the file and exit.

```
jalene@workstation: ~/CPE212_jalene
GNU nano 7.2 README.md *
# CPE212_jalene

hello world
```

- i. Use the `git status` command to display the state of the working directory and the staging area. This command shows which changes have been staged, which haven't, and which files aren't being tracked by Git. Status output does not show any information regarding the committed project history. What is the result of issuing this command?

```
jalene@workstation:~/CPE212_jalene$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

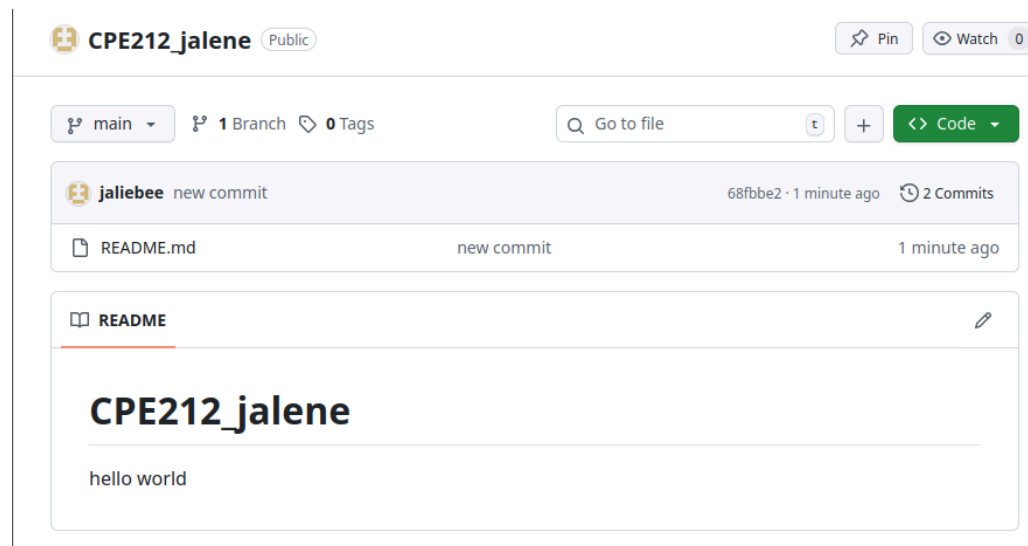
no changes added to commit (use "git add" and/or "git commit -a")
jalene@workstation:~/CPE212_jalene$
```

- j. Use the command `git add README.md` to add the file into the staging area.
- k. Use the `git commit -m "your message"` to create a snapshot of the staged changes along the timeline of the Git projects history. The use of this command is required to select the changes that will be staged for the next commit.
- l. Use the command `git push <remote><branch>` to upload the local repository content to GitHub repository. Pushing means to transfer

commits from the local repository to the remote repository. As an example, you may issue *git push origin main*.

```
jalene@workstation:~/CPE212_jalene$ git add README.md
jalene@workstation:~/CPE212_jalene$ git commit -m "new commit"
[main 68fbbe2] new commit
1 file changed, 3 insertions(+), 1 deletion(-)
jalene@workstation:~/CPE212_jalene$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 271 bytes | 271.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:jaliebee/CPE212_jalene.git
b3a7d2f..68fbbe2  main -> main
jalene@workstation:~/CPE212_jalene$
```

- m. On the GitHub repository, verify that the changes have been made to README.md by refreshing the page. Describe the README.md file. You can notice how long the last commit is. It should be some minutes ago and the message you typed on the git commit command should be there. Also, the README.md file should have been edited according to the text you wrote.



Reflections:

Answer the following:

3. What sort of things have we so far done to the remote servers using ansible commands?

We have used Ansible commands to remotely execute tasks on servers, such as connecting them to SSH and managing configurations without manually logging in. In this activity, ansible allowed us to automate actions across server 1.

4. How important is the inventory file?

The inventory file is very important because it contains all the remote servers that will be managed by ansible. It tells ansible what machines exist, their IP addresses and hostnames and also, how to connect to them.

Conclusions/Learnings:

From this activity, I learned how SSH Key-Based authentication removes the need for passwords and makes remote access more convenient. Additionally, I learned how to set up Git, clone repositories, make commits, and push changes to GitHub using SSH.